MARPE: Proof-Carrying Open Exploration for GPU Kernel Migration

Yuyang Cai Ao Li Cheng Zhao

Team "Nailong Bisheng"

Abstract

Migrating a production GPU kernel to a new tensor-core generation asks more than whether a new instruction is faster: under the same search budget, does it beat the strongest discovered old-path implementation? We study FlashKDA, comparing Ampere-era HMMA with Blackwell `tcgen05` through MARPE, an attributed integration of multi-agent search and execution with open proposals, typed program IR, scoped experience IR, and proof-carrying evaluation. On B300, a resident-grid policy improves qualified `tcgen05` prepare profiles by 1.013–1.135×. Eight matched-opportunity rounds qualify a 1.076× HMMA dispatch gain and a 1.033× H96 `tcgen05` gain over CAKE, while a load-minimax schedule regresses by 6.17%. A preferred-layout probe gains 22.7% over scalar rematerialization but remains 4.7% behind matched HMMA. The executable frontier is locally saturated under the frozen envelope; a production carrier and common-profile H1/T1/T2 qualification remain open.

1 The migration question

FlashKDA is a recurrent state-space kernel used by Kimi Linear and served through FlashInfer [1–3]. Its official route uses HMMA, the matrix-multiply instruction family introduced with Ampere-class GPUs. Blackwell adds `tcgen05`, whose programming model can expose higher throughput but also changes data movement, tensor-memory use, synchronization, and occupancy constraints [4, 5]. A direct "old instruction versus new instruction" microbenchmark therefore misses the production decision.

The mainline question is:

> For the same FlashKDA contract and the same agent/search budget, does the strongest discovered `tcgen05` implementation justify migration from the strongest discovered HMMA implementation?

Comparing optimized `tcgen05` only with untouched HMMA confounds architecture with optimization effort. We therefore separate five anchors:

- **H0**: official production HMMA baseline;
- **H1**: strongest HMMA result found under the declared budget;
- **T0**: matched `tcgen05` baseline under the same contract and timing protocol;
- **T1**: strongest `tcgen05` result found under the same exploration budget;
- **T2**: best guarded SM100 full-stack route, which may include dispatch and auxiliary-kernel changes.

Figure 1 is the compact challenge statement for presentation capture.

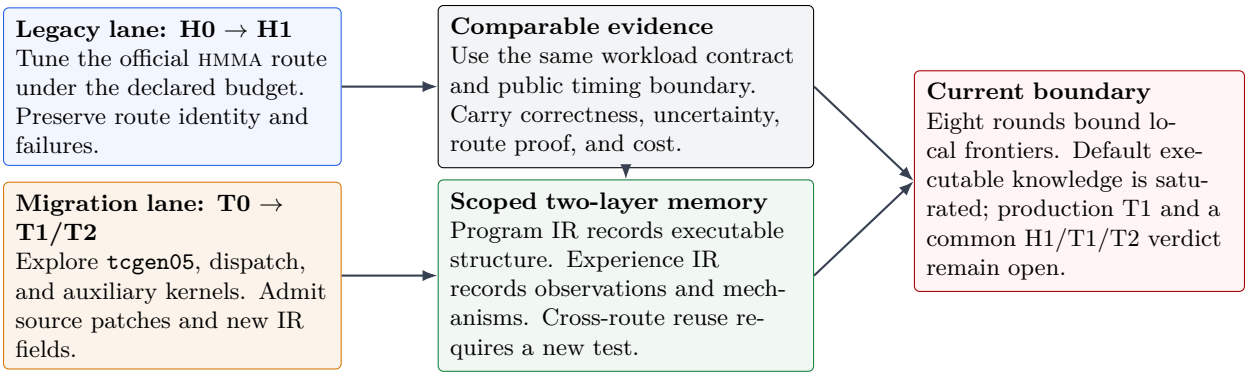

Figure 1: The mainline challenge. Fair migration evidence requires two independently optimized lanes, a shared contract, and scoped transfer of experience. The current typed grammar accelerates execution but cannot define the full proposal space.

2 Why a closed search grammar is insufficient

Early work typed one `tcgen05` coordinate: chunks assigned per prepare CTA. This rejected illegal values and enabled reproducible candidates, but risked confusing what the framework could express with what the kernel could improve.

The routes expose different transformations: HMMA may change warp layout, vector width, or decomposition; `tcgen05` may change tensor memory, roles, clusters, or compiler support. Evidence about `cpc` is not evidence against these families.

MARPE therefore separates an *open proposal plane* from an *executable representation*. Each proposal receives one of four dispositions:

1. `executable_existing`: the current IR and evaluator can run it;
2. `patch_required`: a bounded source change can make it executable;
3. `needs_ir_extension`: the idea is legitimate but the representation lacks a field or transform;
4. `prototype_first`: the mechanism should be tested by a small diagnostic before integration.

Only malformed or out-of-contract proposals are rejected; representation gaps remain in the queue.

3 Two-layer IR and proof-carrying evaluation

3.1 Program IR

The program layer records the operator contract, route, architecture, source identity, schedule, guards, and compilation parameters. Its execution receipt includes the Python executable, package roots, `sys.path`, resolved modules and hashes, native extensions, selected CUDA sources and headers, target code generation, and relevant environment variables. It detects edits made in one checkout while another module or shared object executes.

Typed constraints enforce legal ranges, guards, and route compatibility before compilation; deterministic serialization gives every candidate a stable ID.

3.2 Experience IR

The experience layer stores each proposal, result, evidence and mechanism status, permitted use, and scope keys such as route, architecture, head count, workload, and phase. A timing win is

an observation; its proposed cause becomes reusable only after phase attribution and a falsifiable transfer test.

Failures keep the same scope: an illegal carrier edit constrains that transformation, and a slower virtual-SM rule constrains those resource footprints. Transfer queries matching scope and mechanism, then validates anew.

3.3 Deterministic gates

Promotion follows fixed gates:

1. establish route and build identity;
2. test output and recurrent final-state correctness;
3. time the complete public-call boundary with state reset;
4. use paired ordering and record raw samples;
5. estimate uncertainty and apply the registered decision rule;
6. attribute the physical phase before writing a transferable mechanism;
7. run a fresh held-out or prospective test before strengthening the claim.

Agents supply hypotheses; deterministic gates decide the permitted claim. Search throughput cannot replace route verification, state correctness, or comparable measurement [6–11].

4 Qualified `tcgen05` case study

4.1 Runtime profile and schedule

The recurrent KDA call is described by a runtime profile

$$P = (H, D, d_v, B, T, \tau, \mathcal{L}, S, R),$$

where H is the head count, D and d_v are feature dimensions, B is batch size, T is total token count, τ is dtype, $\mathcal{L}$ is the sequence-length profile, S is the physical SM count, and R is the observed resident prepare-CTA count per SM. The studied Blackwell route launches a prepare kernel followed by a dependency-carrying chain kernel. The schedule parameter `cpc` controls how many chunks each prepare CTA processes.

Let W denote total chunks. For a fixed H, the prepare grid first fits in one resident wave when

$$\texttt{cpc}^*(W, H, S, R) = \left\lceil \frac{W}{\lfloor SR/H \rfloor} \right\rceil.$$

This equation is a physical hypothesis, not merely a fitted threshold. It predicts how the useful `cpc` changes with work, head count, device capacity, and resource footprint.

4.2 Measurement protocol

Qualified measurements use the production public API on B300 compiled for `sm_103a`. Every row checks output and recurrent final state, times the complete call with state reset, alternates candidate and baseline, and reports bootstrap intervals [12]. CUPTI attributes phases [13]; Nsight Compute checks resource assumptions [14].

Table 1: Qualified B300 evidence for the `tcgen05` prepare policy. "Prospective" means the policy selected `cpc` before candidate timing.

Stage	Profiles	Speedup	Evidence
Development	fixed, mixed	1.0369–1.0807	paired CI
Mechanism	4 profiles	97.49–98.86%	CUPTI attribution
Work scale	4 profiles	1.0209–1.0859	predicted winners
Prospective	4 profiles	1.0133–1.1354	adj. LB 1.0129

4.3 Evidence ladder

Table 1 summarizes the evidence. Moving `cpc` from 4 to 9 improved two development calls by 1.0369× and 1.0807×; CUPTI attributed 97.49–98.86% of the reduction to prepare, supporting the wave mechanism.

The model predicted `cpc` values 5, 9, and 17 for $W = 256, 512, 1024$ at $H = 12$, $S = 148$, $R = 5$; preregistered screens selected each winner and matched-work confirmations gained 1.0209–1.0859×. It then prospectively chose 7 and 13 for $W = 384, 768$; four profiles gained 1.0133–1.1354×, with weakest adjusted lower bound 1.0129. The capacity relation, rather than the constant 9, transferred.

Short or highly split profiles do not benefit when prepare already fits or another phase dominates, so the policy remains guarded to its measured domain.

5 Eight-round dual-lane campaign

We ran eight falsification-first rounds with separate HMMA and tcgen05 queues. Rounds 1–7 gave each lane one challenger and two balanced GPU blocks; positive screens received four-block qualification. Round 8 allowed one structural hypothesis, one same-mechanism repair, three compiles, two correctness attempts, and two screen blocks per lane. The ledger matches present opportunities, not historical maturity; differing Round 8 scopes prohibit cross-lane latency comparison.

HMMA exposed a piecewise frontier. V64 beat V128 by 1.2172× and 1.1723× at H12 nseq 6, but fell below the 3% gate at nseq 7–8. V32 beat V64 by 1.0764× at balanced nseq 3 and 1.0774× on a skewed holdout, yet regressed 3.82% at nseq 4. The result is a CTA-wave crossover, not one best width. Lookahead depth 3 was bitwise correct but neutral to depth 2 (0.99998×).

tcgen05 rejected a virtual-152-CTA rule on H64 and H96, then qualified H96 physical-148/s173 at 1.0330× over Cake. Grid147 gained 0.34%, while TAIL4 and MINIMAX172 regressed. A constructively optimal maximum load of 169 still regressed by 6.17%, proving that load minimax is not this carrier's latency objective.

Round 8 paired a full HMMA forward screen with a smaller tcgen05 V16 mechanism probe. Moving logical-B rematerialization outside the repeated lifecycle reduced registers 39 to 32 and static shared memory 9740 to 9228 bytes, preserved bitwise correctness, and ran 1.227× faster than scalar tcgen05 but 0.955× as fast as resident HMMA. It isolates layout cost without establishing a production P3/P4 or TMEM carrier.

An H96 request silently fell back to Cake, a fresh-seed H12 Cake arm failed correctness, and an HMMA call used an incompatible binding. The rows were excluded or repaired within budget, motivating first-class module paths, hashes, symbols, launches, and failures.

Table 2: Key dual-lane evidence. Q: four-block qualification; S: scoped screen or mechanism probe.

Lane	Intervention	Result	Status
HMMA	V32/V64, balanced nseq 3	1.0764×	Q
HMMA	V32/V64, balanced nseq 4	0.9618×	S
HMMA	lookahead L3/L2	0.99998×	S
tcgen05	H96 s173/CAKE	1.0330×	Q
tcgen05	load-optimal 169/default	0.9383×	S
tcgen05	preferred/scalar probe	1.227×	S
tcgen05	preferred/HMMA probe	0.955×	S

6 Agent architecture and research memory

Open-source provenance. MARPE is an integration and domain adaptation, rather than a multi-agent system implemented from scratch. Its search plane reuses PIKE-B's independent initial proposals, parallel branches, repair agents, candidate allocation, and top-branch selection [15]. Its execution and evidence plane reuses Atrex Kernel Agent's isolated GPU execution, NVIDIA profiling, Git-worktree episodes, resumable journals, and incumbent/candidate ABBA verification [16]. The artifact vendors audited snapshots at PIKE revision `fd5721e9` under MIT and Atrex revision `d643fb5b` under Apache-2.0, with upstream notices retained.

Contribution and experimental boundary. The work introduced here is the FlashKDA-specific control plane: separate HMMA and tcgen05 Program IRs, scope-keyed Experience IR, a matched-opportunity budget protocol, and proof-carrying candidate promotion. These additions keep route-specific legal transformations separate while making correctness, execution identity, uncertainty, and permitted claim scope authoritative. The present experiment is a narrow-width dual-branch loop, with one new challenger per lane in Rounds 1–7 and one structural hypothesis per lane in Round 8. We did not run a matched single-agent ablation, so the results do not establish that multi-agent search outperforms a single agent; they establish the behavior of this attributed integration on the migration case study.

MARPE uses roles to diversify hypotheses, not to vote on truth. A scout inspects source and profiles bottlenecks. A proposer emits typed or source-identified edits. A falsifier searches for counterexamples and route mistakes. An evaluator runs deterministic gates. A synthesizer writes scoped experience entries and updates the proposal queue. Roles may operate concurrently, while promotion and memory writes remain serialized by candidate identity.

The campaign allocates matched opportunity caps to the two lanes: proposal, repair, compile, correctness, screen, and qualification slots are frozen before execution. Failed builds and wrong-route runs consume their slots. Within a lane, schedule parameters, source transformations, dispatch policies, and representation extensions compete in one ledger. Search cost remains separate from historical code maturity and code size; equal present-day opportunities cannot erase different starting maturity. When evidence scopes differ, as in a public extension and a mechanism probe, the ledger compares exploration opportunity rather than latency.

Experience reuse follows three rules. First, transfer invariants such as route verification, state reset, and execution receipts broadly. Second, transfer mechanism hypotheses only when their variables exist on the destination route. For example, resident-wave reasoning may transfer, while the fitted H12 `cpc` value does not. Third, treat contradictions as scope refinement. When H12 and H96 disagree, memory should split by resource footprint or route phase instead of overwriting one result.

This structure avoids two common agent failures. A memory system that stores only winners repeats unsafe attempts because it loses the boundary of legality. A memory system that globalizes every failure suppresses new ideas because one route's constraint becomes another route's false prohibition. Scoped program and experience IR provide a middle ground: agents inherit concrete evidence while the proposal plane stays open.

7 What the current evidence establishes

The evidence supports five claims. First, runtime profile belongs in the optimization representation: one compiled operator needs different physical schedules and vector widths at different work scales. Second, proof-carrying evaluation catches route fallback, stale modules, incompatible bindings, and correctness failures that timing alone misses. Third, mechanism-backed memory transfers better than constants: the resident-grid equation predicted new work scales, whereas virtualizing the SM count failed. Fourth, legacy exploration is necessary because H0 was materially improvable and its dispatch frontier was piecewise. Fifth, a theoretical optimum in one IR coordinate need not optimize latency: the exact load-minimax schedule was substantially slower.

Within the frozen B300, existing-carrier envelope, an independent proposal audit found no new high-value executable family. HMMA vector-width and lookahead hypotheses have scoped boundaries; tcgen05 grid, task-grouping, load-minimax, and one-time preferred-layout hypotheses have direct evidence. We call this *default-knowledge high-value proposal saturation*, not a hardware or algorithmic optimum. Cheap unmotivated variants remain possible, but repeating them would not add a new mechanism.

The headline migration verdict remains open because the strongest tcgen05 structural idea now requires information outside the envelope: a lane-verified P3/P4 preferred-layout producer, an explicit BF16 rounding boundary, or cross-phase TMEM residency. The tcgen05 probe cannot stand in for production T1, and its latency cannot be compared with the HMMA public-forward screen. The next decisive work is evidence closure: build that carrier, freeze common profiles and the public-call boundary, and qualify H1, T1, and T2 with paired uncertainty. A global optimum would additionally require a tight latency lower bound.

8 Related work

TensorIR, Ansor, and MetaSchedule use structured program representations to define legal search spaces [17–19]. Triton and CUTLASS expose reusable GPU scheduling and tensor-core abstractions [20, 21]. Stream-K shows why work decomposition and occupancy interact with shape and device scale [22]. MARPE focuses on an operational gap: a production migration may require proposals outside the current grammar, and every reported win must remain tied to the route, environment, and public-call contract that produced it.

Recent kernel agents broaden candidate generation through language models, evolutionary search, and iterative feedback [7–10]. Our contribution is complementary. We use agents to explore an open proposal plane, while deterministic evidence gates and scoped memory control what becomes reusable knowledge. This separation is especially useful when two architectural routes have different legal transformations and maturity.

9 Conclusion

The FlashKDA migration question cannot be resolved by optimizing `tcgen05` against an untouched HMMA baseline. It requires two independently optimized lanes, a shared contract, and evidence that records exactly what executed. MARPE combines an open proposal plane with typed program IR, scoped experience IR, and proof-carrying promotion. Eight B300 rounds qualified a profile-dependent HMMA dispatch improvement, rejected a mathematically optimal but slower tcgen05 load schedule, and isolated a preferred-layout cost without overstating a microprobe as T1. Existing-carrier high-value proposal generation is now saturated under the frozen envelope. The remaining work changes the representation itself: construct the production tcgen05 carrier, then earn the common-profile H1/T1/T2 verdict.

References

[1] Kimi Team. Kimi linear: An expressive, efficient attention architecture, 2025. URL `https://arxiv.org/abs/2510.26692`.

[2] Moonshot AI. FlashKDA: Efficient kernel for kimi delta attention. GitHub repository, 2025. URL `https://github.com/MoonshotAI/FlashKDA`.

[3] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. Flashinfer: Efficient and customizable attention engine for LLM inference serving. In *Eighth Conference on Machine Learning and Systems*, 2025. URL `https://openreview.net/forum?id=RXPofAsL8F`.

[4] NVIDIA. *CUDA Programming Guide*. NVIDIA Corporation, 2026. URL `https://docs.nvidia.com/cuda/cuda-programming-guide/`.

[5] NVIDIA. *NVIDIA Blackwell Tuning Guide*. NVIDIA Corporation, 2026. URL `https://docs.nvidia.com/cuda/blackwell-tuning-guide/`.

[6] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Re, and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels?, 2025. URL `https://arxiv.org/abs/2502.10517`.

[7] Anjiang Wei, Allen Nie, Thiago S. F. X. Teixeira, Rohan Yadav, Wonchan Lee, Ke Wang, and Alex Aiken. Improving parallel program performance with LLM optimizers via agent-system interface, 2024. URL `https://arxiv.org/abs/2410.15625`.

[8] Anjiang Wei, Tianran Sun, Yogesh Seenichamy, Hang Song, Anne Ouyang, Azalia Mirhoseini, Ke Wang, and Alex Aiken. Astra: A multi-agent system for GPU kernel performance optimization, 2025. URL `https://arxiv.org/abs/2509.07506`.

[9] Kaiming Cheng, Laura Wang, Jack Khuu, Mark Saroufim, Wenyuan Chi, Jiannan Wang, and Joe Isaacson. KernelAgent: Hardware-guided GPU kernel optimization via multi-agent orchestration. PyTorch Blog, 2026.

[10] Joyjit Kundu, Ben Stoffelen, Kaili Wang, Peter Vrancx, and Ludovic Denoyer. KernelArc: A multi-agent framework for GPU kernel optimization, 2026. URL `https://arxiv.org/abs/2608.17071`.

[11] Yunxiang Zhang, Ping Yu, Jianyu Wang, Max Fan, Julian Reed, Azalia Mirhoseini, and Will Su. KernelBench-Verified: Do LLM-generated kernels actually beat PyTorch?, 2026. URL `https://arxiv.org/abs/2607.16241`.

[12] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.

[13] NVIDIA. CUPTI activity api, 2026. URL https://docs.nvidia.com/cupti/api/group__CUPTI__ACTIVITY__API.html.

[14] NVIDIA. Nsight Compute profiling guide, 2026. URL https://docs.nvidia.com/nsight-compute/ProfilingGuide/.

[15] Kirill Nagaitsev, Luka Grbcic, Samuel Williams, and Costin Iancu. Optimizing PyTorch inference with LLM-based multi-agent systems, 2025. URL https://arxiv.org/abs/2511.16964. PIKE open-source repository: https://github.com/pike-project/pike.

[16] Lingyun Yang, Yuxiao Wang, Shenghao Liang, Linfeng Yang, Daocheng Ying, Chunbo You, Rui Zhang, Luping Wang, Yinghao Yu, Guodong Yang, and Liping Zhang. Are LLM-generated GPU kernels production-ready? a trace-driven benchmark and optimization agent, 2026. URL https://arxiv.org/abs/2607.14541. Atrex Kernel Agent open-source repository: https://github.com/alibaba/atrex-kernel-agent.

[17] Siyuan Feng, Bohan Hou, Hongyi Jin, Wuwei Lin, Junru Shao, Ruihang Lai, Zihao Ye, Lianmin Zheng, Cody Hao Yu, Yong Yu, and Tianqi Chen. TensorIR: An abstraction for automatic tensorized program optimization. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems*, 2023. URL https://dl.acm.org/doi/10.1145/3575693.3576933.

[18] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor: Generating high-performance tensor programs for deep learning. In *USENIX Symposium on Operating Systems Design and Implementation*, 2020. URL https://www.usenix.org/conference/osdi20/presentation/zheng.

[19] Junru Shao, Xiyou Zhou, Siyuan Feng, Bohan Hou, Ruihang Lai, Hongyi Jin, Wuwei Lin, Masahiro Masuda, Cody Hao Yu, and Tianqi Chen. Tensor program optimization with probabilistic programs. In *Advances in Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2205.13603.

[20] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019. URL https://doi.org/10.1145/3315508.3329973.

[21] NVIDIA. CUTLASS: CUDA templates for linear algebra subroutines. Software and documentation, 2026. URL https://github.com/NVIDIA/cutlass.

[22] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. Stream-K: Work-centric parallel decomposition for dense matrix-matrix multiplication on the GPU. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 429–431, 2023. doi: 10.1145/3572848.3577479. URL https://arxiv.org/abs/2301.03598.